

CONSULTAS AVANZADAS EN SPRING DATA JPA

Objetivo

Aprender a crear, estructurar y ejecutar consultas del dominio de negocio sobre bases de datos relacionales utilizando **Spring Data JPA**. Exploraremos en profundidad opciones como la inferencia de consultas, lenguaje JPQL con `@Query`, SQL Nativo, y el manejo de paginación y ordenamiento; siempre utilizando el dominio de nuestro proyecto (*Solicitudes*).

Conceptos Básicos

Bases de Datos Relacionales: Modelado de entidades y relaciones mediante tablas (H2, PostgreSQL, MySQL).

Inferencia (Method Derivation): Generación automática del código SQL mediante la convención de nombres del framework Spring Data.

JPQL (Java Persistence Query Language): Lenguaje orientado a objetos que permite escribir consultas entendiendo Clases Reales en lugar de Tablas de Base de Datos.

Paginación: Estrategia de recuperación segmentada de grandes volúmenes de datos.

Contextualización Teórica

Spring Data JPA

Como recordamos de la Guía 10, Spring Data JPA actúa como una súper capa de abstracción sobre JDBC e Hibernate reduciendo dramáticamente el código repetitivo (*boilerplate*), especialmente para operaciones de

búsqueda. Para expresar su potencial, ofrece tres mecanismos fundamentales de consultas:

Tipos de Consultas

1. Inferencia de Métodos (Query Methods)

El comportamiento clásico y más impactante de Spring Data JPA es generar consultas SQL en tiempo de ejecución **solo interpretando el nombre del método de la interfaz**.

La sintaxis del framework permite construir reglas de igualdad, ordenamiento e inspección de Value Objects (objetos embebidos).

Ejemplo en el Proyecto de Solicitudes:

```
public interface SolicitudJpaDataRepository extends JpaRepository<SolicitudEntity, Long> {  
  
    // Igualdad Básica  
    List<SolicitudEntity> findByEstado(EstadoSolicitudEnum estado);  
  
    // Atributos Encadenados (And, Or)  
    List<SolicitudEntity> findByTipoAndEstado(TipoSolicitudEnum tipo, EstadoSolicitudEnum estad  
  
    // Búsqueda por patrón (LIKE automático)  
    List<SolicitudEntity> findByDescripcionContainingIgnoreCase(String keyword);  
  
    // Navegando por Objetos Embebidos (PrioridadEmbeddable -> nivel)  
    List<SolicitudEntity> findByPrioridad_Nivel(NivelPrioridadEnum nivel);  
  
    // Ordenamiento Dinámico  
    List<SolicitudEntity> findByPropietarioldOrderByFechaCreacionDesc(String propietarioId);  
  
    // Operaciones Analíticas y Booleanas  
    long countByEstado(EstadoSolicitudEnum estado);  
    boolean existsByCodigo(String codigo);  
}
```

Palabras clave comunes:

Operador	Equivalente SQL	Ejemplo
And , Or	WHERE x = ? AND y = ?	findByEstadoAndTipo(...)
Between	WHERE x BETWEEN ?1 AND ?2	findByFechaCreacionBetween(...)
LessThan , GreaterThan	WHERE x < ? , WHERE x > ?	findByFechaCreacionLessThan(...)

Operador	Equivalente SQL	Ejemplo
In	WHERE x IN (?, ?)	findByEstadoIn(...)
Containing	WHERE x LIKE '%?%'	findByDescripcionContaining(...)
OrderBy[Attr]Desc	ORDER BY x DESC	findByEstadoOrderByFechaCreacionDesc(...)

?? [Referencia completa de palabras clave de Spring Data JPA](#)

2. Consultas con JPQL (@Query)

La derivación de nombres es idónea para consultas simples, pero si la condición involucra reglas complejas de negocio, varios parámetros OR, u optimizaciones de carga (JOIN FETCH), los nombres de los métodos pueden tornarse ilegibles (ej. *findByEstadoOrPrioridadAndNoAsignadoAnd...*).

Aquí interviene **JPQL**. A diferencia de SQL, JPQL opera directamente sobre nuestras Entidades y sus Atributos (SolicitudEntity y descripcion), ignorando los nombres reales de las columnas en la base de datos (Ej desc_tmp o tabla solicitudes). La lógica queda divorciada de la persistencia particular y sigue unificada con el código Java.

Ejemplo:

```
public interface SolicitudJpaDataRepository extends JpaRepository<SolicitudEntity, Long> {

    // JPQL clásico con Parámetros Indexados (?1, ?2)
    @Query("SELECT s FROM SolicitudEntity s WHERE s.estado = ?1 OR s.tipo = ?2") List<SolicitudEntity>
    buscarPorEstadoOTipo(EstadoSolicitudEnum estado, TipoSolicitudEnum ti

    // JPQL con Parámetros Nombrados (más legible cuando hay muchos parámetros) @Query("SELECT s FROM
    SolicitudEntity s WHERE s.responsableId IS NULL AND s.prioridad.nivel List<SolicitudEntity>
    buscarSinAsignarPorPrioridad(@Param("nivelPrioridad") NivelPrioridadE

    // Uso del operador IN
    @Query("SELECT s FROM SolicitudEntity s WHERE s.estado IN (:estados)")
        List<SolicitudEntity> buscarPorVariosEstados(@Param("estados") List<EstadoSolicitudEnum> es

    // Evitando problemas de rendimiento (N+1) forzando el join sobre colecciones perezosas @Query("SELECT s FROM
    SolicitudEntity s LEFT JOIN FETCH s.historial WHERE s.codigo = :codig Optional<SolicitudEntity>
    buscarSolicitudConHistorial(String codigo);

}
```

⚠ Importante: Con JPA y arquitecturas DDD (donde los repositorios devuelven los Agregados enteros), traer relaciones pesadas solo cuando se requiere usando JOIN FETCH marca una gran diferencia de rendimiento.

3. Consultas Nativas (Native Queries con SQL Puro)

Excepcionalmente, te puedes topar con una situación que involucre funcionalidad hiperespecífica de un motor (funciones geográficas de Postgres o Hints de base de datos) imposibles de mapear en JPQL. En tal caso se recurre al parámetro `nativeQuery = true` de la anotación `@Query`. Al usar esto, regresas nuevamente a los nombres reales de las columnas en BD.

```
@Query(  
    value = "SELECT COUNT(*), estado FROM solicitudes GROUP BY estado",  
    nativeQuery = true  
)  
List<Object[]> reporteAgrupacionPorEstadoNativo();
```

Nota: Limita el uso de Native Queries en lo más posible, pues rompen la compatibilidad si algún día cambias de tipo de gestor de Base de Datos.

Paginación y Ordenamiento

A medida que crece el sistema, es una mala práctica cargar listados masivos. Las interfaces Spring proveen objetos genéricos súper potentes para esto: `Pageable` para peticiones y `Page<T>` para retorno.

La inferencia y el `@Query` soportan la paginación. Modificar un método es tan simple como agregarlo al final de su firma:

```
// Devuelve una página y meta-información (total elementos, número de página) Page<SolicitudEntity>  
findByResponsableId(String responsableId, Pageable pageable);
```

```
// Con una constante @Query  
@Query("SELECT s FROM SolicitudEntity s WHERE s.estado = :estado")
```

```
Page<SolicitudEntity> buscarPorEstadoPaginado(@Param("estado") EstadoSolicitudEnum estado, Page Desde el punto de
```

vista del Servicio (Adaptador / Casos de Uso), su uso sería:

```
// Le indicamos a Spring que queremos consultar los recursos de la página 0 y solo con tamaño 1 Pageable paginaSolicitada = PageRequest.of(0, 10, Sort.by("fechaCreacion").descending());
```

```
// Enviamos al repositorio el pageable, que lo procesará insertando un 'LIMIT' / 'OFFSET' invis Page<SolicitudEntity> resultado = repository.findByResponsableId("USER1", paginaSolicitada);
```

```
long totalViendo = resultado.getTotalElements(); // Elementos totales de la paginación (más all int hojas = resultado.getTotalPages());
```

```
List<SolicitudEntity> items = resultado.getContent(); // Tu arreglo de solo las 10 entidades
```

Comparativa de Enfoques

Enfoque	Ventajas	Desventajas
Inferencia de Métodos	✓ Altamente legible y limpio ✓ Cero código SQL	✗ Nombres de métodos demasiado largos si son complejas, inmanejable para JOINS grandes.
JPQL (@Query)	✓ Flexible y orientado a objetos ✓ Protege ante cambios directos de esquema en la BD	✗ Tienes que escribir pseudo-SQL manualmente a veces perdiendo agilidad.
SQL Nativo (nativeQuery)	✓ Acceso a utilidades específicas del motor ✓ Optimización absoluta en ciertos scripts	✗ Acoplamiento total a la BD y a cómo nombramos las columnas físicas perdiendo el mapeo de objetos de hibernate.

Ejercicios Prácticos

1. Cree una consulta generada puramente por inferencia que obtenga la lista de *Solicitudes* realizadas para reportar un problema homologación (TipoSolicitudEnum.HOMOLOGACION) ordenadas por prioridad de mayor a menor o fecha de creación.

Prueba conceptual: Defina la interfaz `findByTipoOrderByPrioridad_NivelDesc(...)`

2. Recree el método anterior pero en su reemplazo escriba explícitamente su instrucción en un `@Query` (JPQL). Fíjese en la sintaxis requerida para acceder al *embedded value object*.
3. Cree una consulta JPQL que permita obtener solicitudes filtrando por `Codigo` , o de forma opcional por la identificación de Solicitante propietarioId . Cuando pase el código, este debe tener prioridad, cuando no lo pase (ej, sea equivalente a `%`), traiga el filtro de propietarioId. (Opcionalmente, use

validaciones en el String del JPQL WHERE (:codigo IS NULL OR s.codigo = :codigo) de modo que ignore el filtro de código).

4. Ejercicio de Paginación Avanzado:

Cree en su adaptador `SolicitudJpaAdapter` un método que reciba la página actual, el tamaño y que llame internamente a su Repositorio usando la Inferencia de Nombres para buscar cualquier solicitud con estado distinto a ELIMINADO devolviendo en el objeto nativo del dominio solamente el envoltorio del fragmento.

Para la próxima clase

Investigue sobre las pruebas unitarias y su importancia en sistemas acoplables como este.

Leer:

i. [Spring Boot Testing Oficial](#)

Guía oficial de Spring Boot sobre testing (incluye unitarios, integración y slices de Capas).

ii. [Mockito + Spring Boot](#)

Documentación de Mockito (mocking general de dependencias de interfaces).

REFERENCIAS BÁSICAS

Documentación Oficial

[Spring Data JPA Reference](#)

[Keywords JPQL por Nombres \(Supported keywords inside method names\)](#)

[Proyecciones en Spring Data \(Opcional Avanzado\)](#)

Criterios de Evaluación

Se espera que el estudiante tras comprender esta etapa:

1. Reconozca cuándo se debe optar por un Nombre de Inferencia Limpio y cuándo se le cede el manejo a JPQL para evitar métodos incomprensibles.
2. Identifique cómo consultar datos encajados (Value Objects) en Spring Data JPA a nivel Relacional.
3. Configure parámetros Paginados eficientes e incorpore la limitación a nivel de SQL.